University of Sialkot

Department of Software Engineering

Faculty of Software Engineering

Web Engineering

Guiding Principles: Smart Diet Planner(TandooriFit)

Submitted To:

Sir Museb Khalid

Dated: 30/06/2026

By:

Muhammad Jasim 23010101-054

Atiqa Asif 23010101-016

Introduction

Smart Diet Planner is a client-side web application designed to help individuals plan and manage a healthy diet in a simple and personalized way. It allows users to calculate essential health metrics such as BMI, BMR, TDEE, and daily calorie requirements based on their age, gender, body measurements, activity level, and fitness goals. Based on these results, the application assists users in creating a structured daily meal plan with macro-nutrient targets and flexible meal distribution, making diet planning more practical and user-friendly.

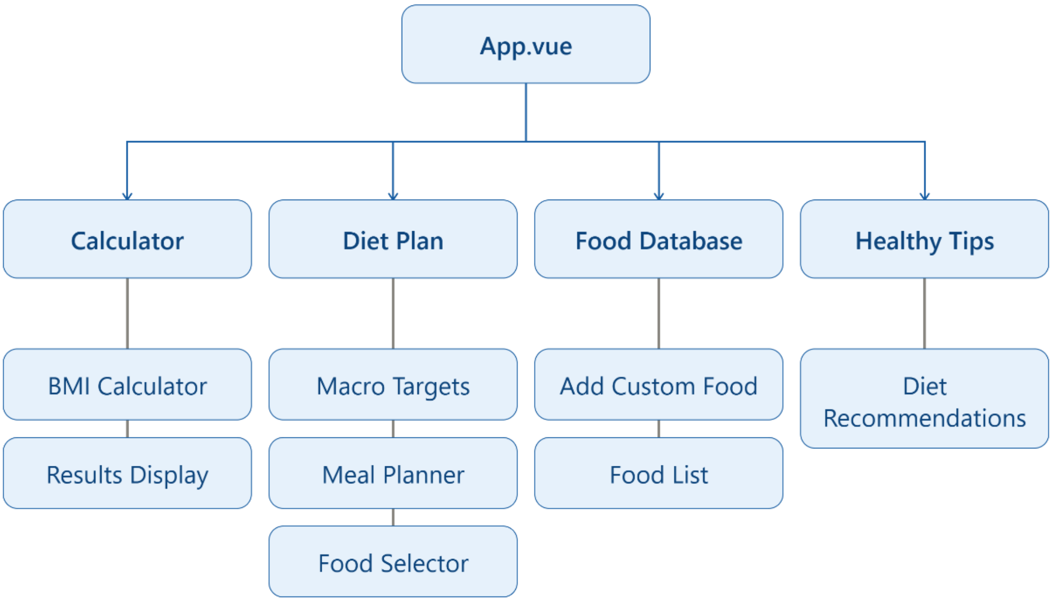
The application is useful for users who want a clear and guided approach to nutrition without complex manual calculations. Many people find it difficult to estimate calories and macros or to design balanced meals that align with their fitness goals. Smart Diet Planner addresses this problem by providing automated calculations, customizable meal planning, and a flexible food database that can be adapted or extended in future versions. By focusing on simplicity, clarity, and modular design, the application lays a strong foundation for future enhancements such as online data storage, user accounts, and dynamic food databases.

Project Features

- **BMI Calculator**
  Calculates Body Mass Index using user-provided height and weight and categorizes the result to help users understand their current health status.
- **Daily Calorie Calculator**
  Computes daily calorie requirements based on BMR, activity level, and fitness goals such as weight maintenance, loss, or gain.
- **Meal Planner**
  Generates a structured daily meal plan according to calorie targets, selected diet style, and number of meals per day.
- **Macro-Nutrient Breakdown**
  Provides detailed information about daily protein, carbohydrate, and fat requirements to support balanced nutrition.
- **Flexible Food Database**
  Allows users to manage food items and nutritional values, with support for modifying or extending the food list as needed.
- **Diet Recommendations**
  Offers healthy dietary tips and guidance to promote better eating habits and informed food choices.
- **User-Friendly Dashboard**
  Presents calculated results and meal plans in a clear and organized interface for easy monitoring.

- **Responsive Single Page Application**
  Ensures smooth navigation and usability across different screen sizes using client-side routing.

Architecture Tree:



Vue Router

- / → **CalculatorView**
  (BMI, BMR, TDEE, and daily calorie calculator)
- /plan → **PlanView**
  (Daily meal planner with macro breakdown and meal management)
- /database → **DatabaseView**
  (Food database management with add/edit and macro estimation)
- /tips → **TipsView**
  (Healthy diet tips and nutrition guidance)

Components

- **App.vue** → Root component that manages layout, navigation, and shared application state
- **BmiCalculator.vue** → Takes user input for BMI, BMR, TDEE, and calorie calculations
- **BmiResults.vue** → Displays calculated health results such as BMI category and daily calorie needs
- **CalculatorView.vue** → Combines calculator input and results into one page

- **PlanView.vue** → Handles meal planning, meal updates, and macro tracking
- **DatabaseView.vue** → Manages food database and custom food entries
- **TipsView.vue** → Displays healthy diet tips and nutrition guidance
- **macroEstimator.js (utility)** → Automatically estimates food macros based on name keywords
- **router/index.js** → Manages navigation between different views in the SPA

Props and Emits

In Vue.js, **props** and **emits** are used for communication between components.

Props are used to send data from a parent component to a child component. This allows child components to display or use data provided by the parent.

Emits are used when a child component wants to send information or trigger an action in the parent component. It works like sending a message upward.

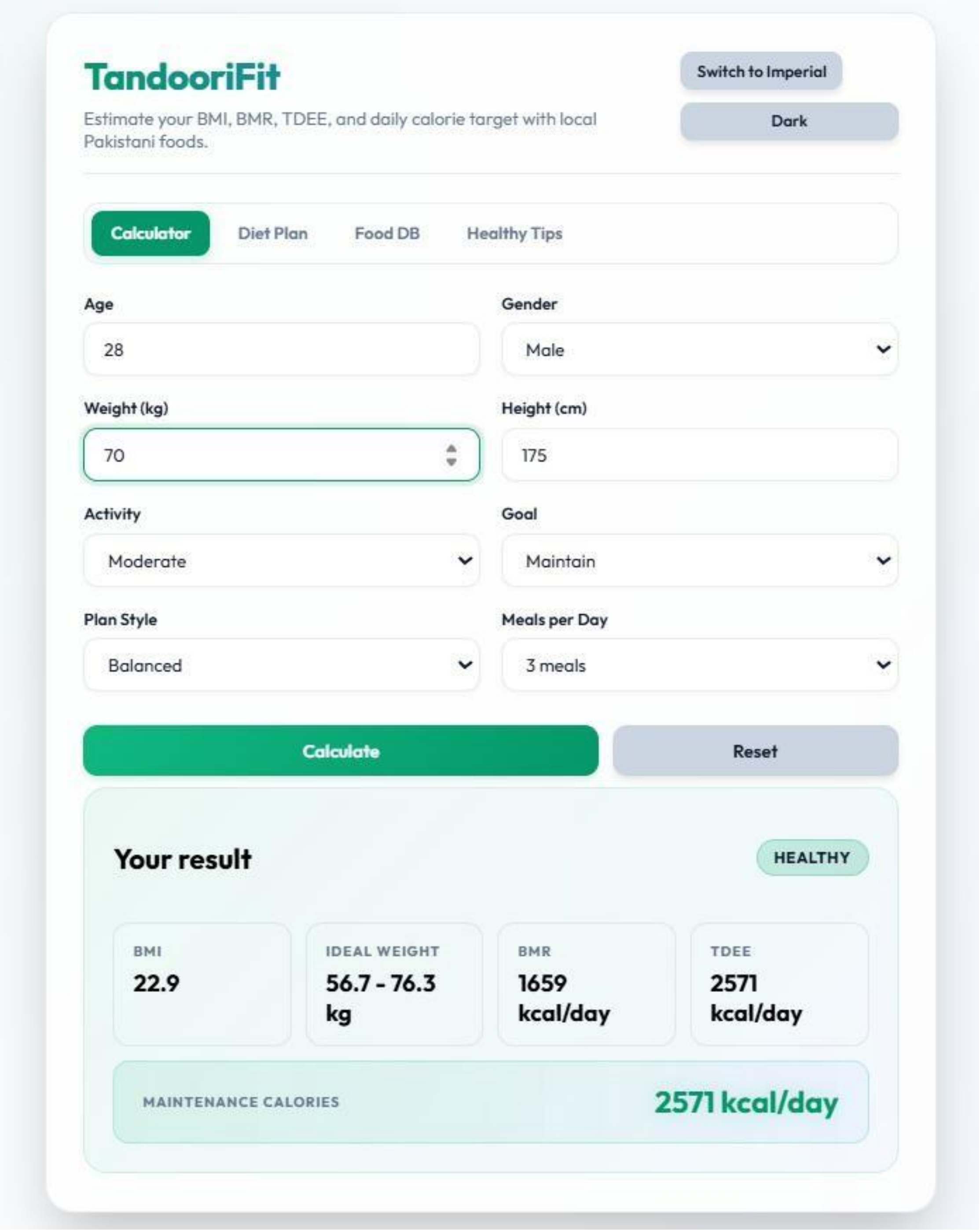
Example code of BmiCalculator Component including props and emitters:

```
const props = defineProps({

  units: String,

  age: Number,

gender: String,

  weightKg: Number,

weightLbs: Number,

heightCm: Number,

feet: Number,   inches:

Number,

  activity: String,

goal: String,

planStyle: String,

  mealsPerDay: Number

});

const emit = defineEmits([

  'update:units', 'update:age', 'update:gender', 'update:weightKg',

  'update:weightLbs', 'update:heightCm', 'update:feet', 'update:inches',
```

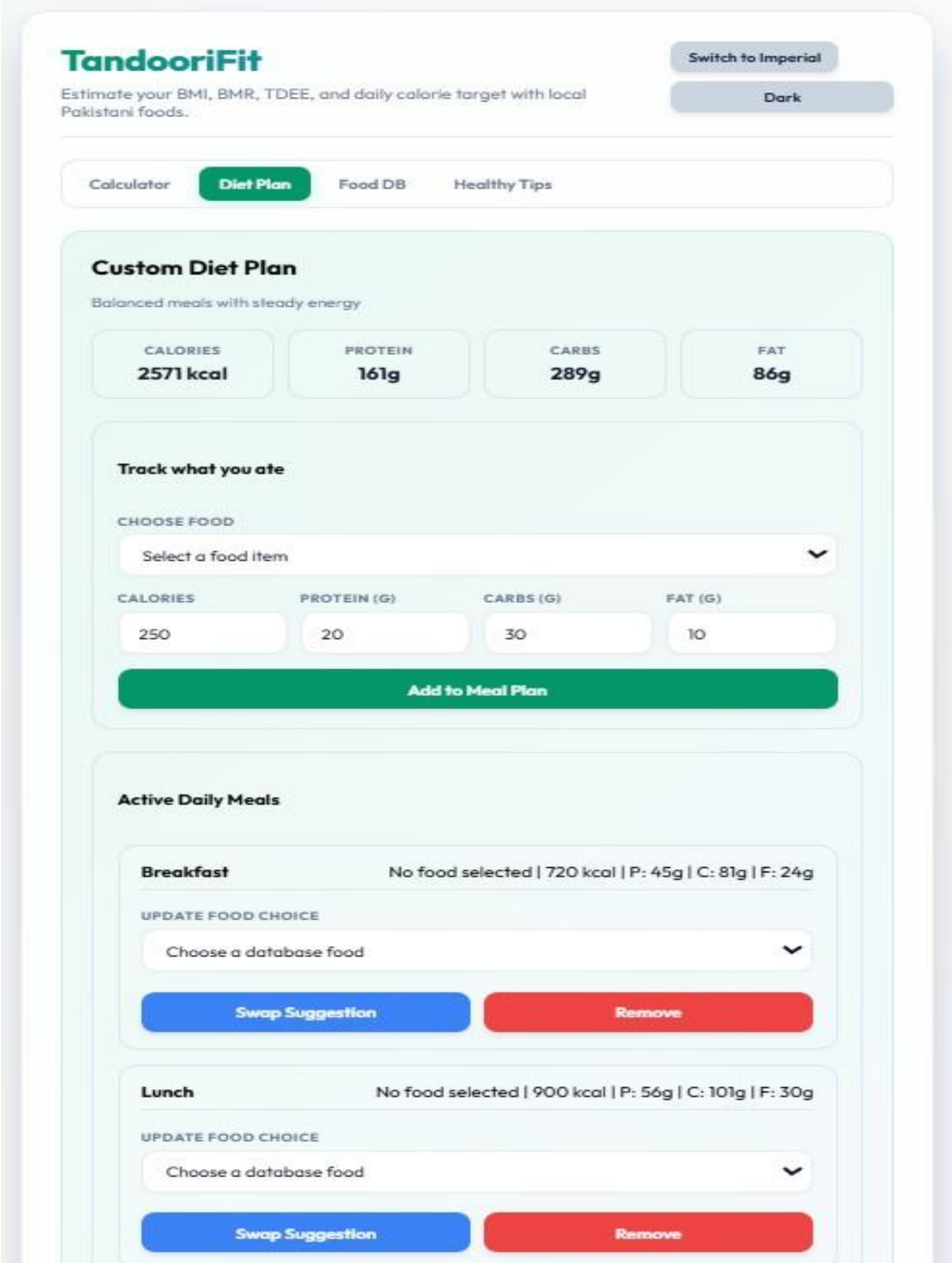
```
  'update:activity', 'update:goal', 'update:planStyle', 'update:mealsPerDay',
  'calculate', 'reset'
]);
```

Screenshots:

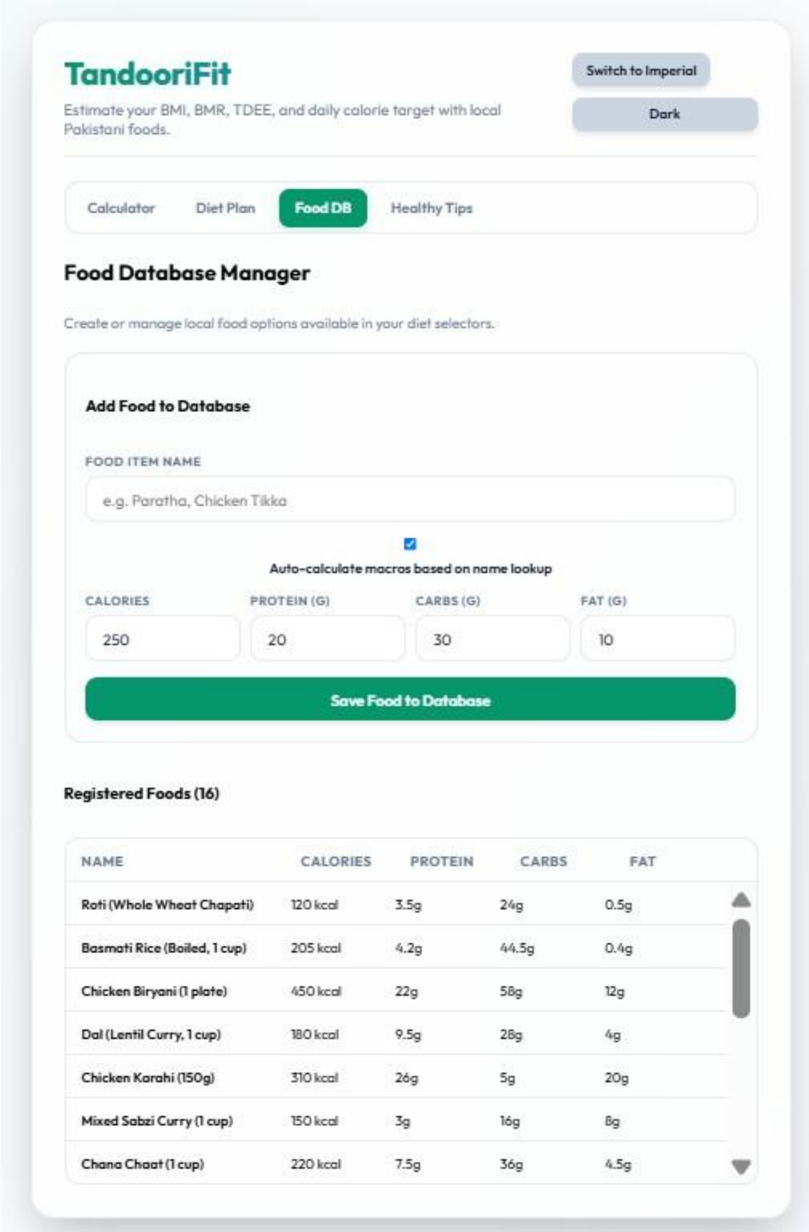
Calculator:



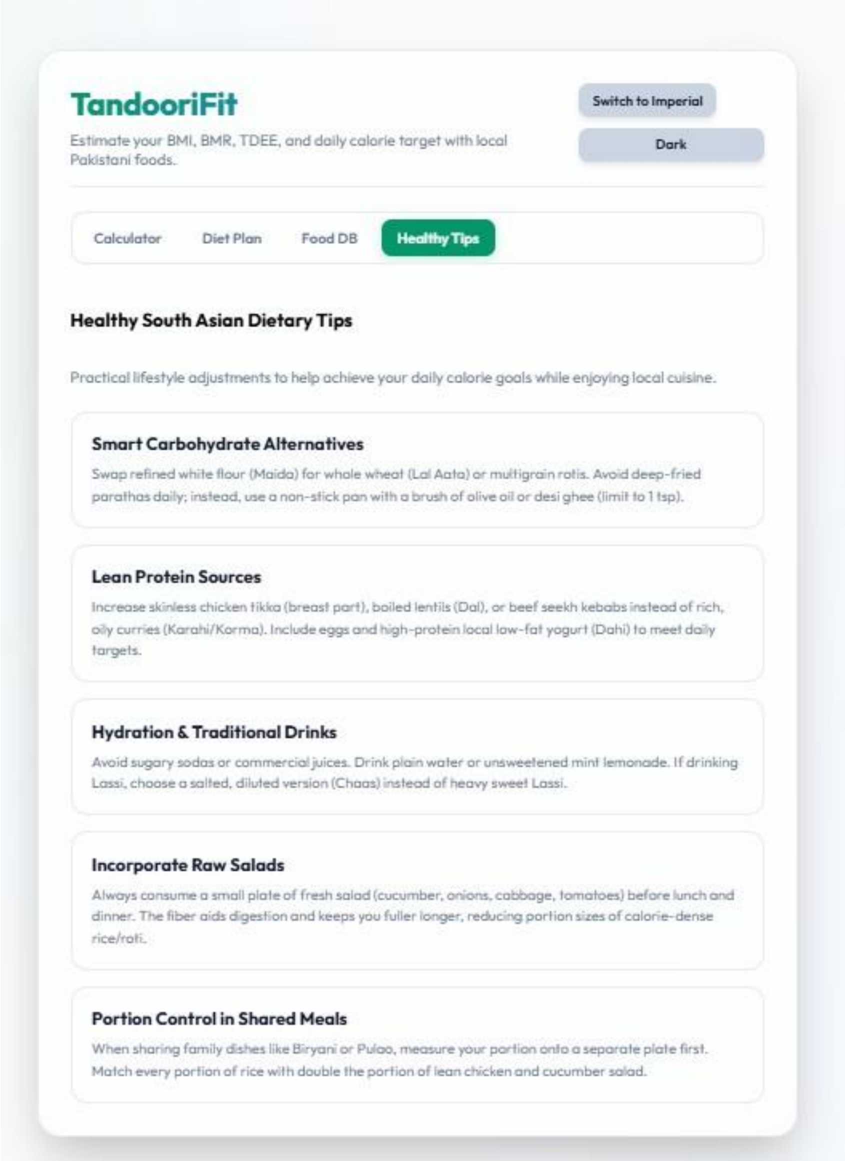
Diet Plan:



Food Database:



Healthy Tips:



Github Repository Link:

https://github.com/jasimjawediqbal/Smart-Diet-Planner

Conclusion

The Smart Diet Planner frontend was developed using Vue 3 and Vite, focusing on a modern and modular single-page application architecture. The system uses Vue Router for smooth navigation between different views, while reusable components ensure a clean and organized code structure. Communication between components is handled using props and emits, following proper parent-child data flow principles. This frontend serves as a strong foundation for the project, and the backend integration will be developed in the next phase to extend its functionality.